

ComboTile Code Documentation

William Lee

A Python library for loading, mosaicking, and analysing Digital Surface Models (DSMs) on both Earth (ALOS) and the Moon, with support for horizon-angle computation, slope mapping, and ENU-frame sky-view analysis.

Module file	<code>terrain_tile.py</code>
Classes	<code>ComboTile</code> , <code>JTile</code> , <code>LunarTile</code>
Language	Python 3.9+

Contents

1	Overview	3
2	Class Hierarchy	3
3	ComboTile — Core Terrain Tile	3
3.1	Constructor	3
3.2	Coordinate Utilities	3
3.2.1	surrounding_tiles	3
3.2.2	get_lon_lat	4
3.2.3	xy_to_lonlat / lonlat_to_xy	4
3.2.4	get_elevation	4
3.3	Gradient and Slope	4
3.3.1	compute_gradient	4
3.3.2	plot_slope	4
3.4	Horizon / Sky-View Analysis	4
3.4.1	max_elev_enu	4
3.5	Visualisation	5
3.5.1	plot_map	5
3.6	Data Export	5
3.6.1	map_table	5
3.6.2	save_data	5
4	JTile — Single-File Earth Tile	5
5	LunarTile — Lunar DEM Tile	6
5.1	Constructor	6
5.2	Overridden Methods	6
5.2.1	get_lon_lat	6
5.2.2	compute_gradient	6
5.2.3	plot_map	6
5.2.4	max_elev_enu	7
6	Key Algorithms	7
6.1	ENU Frame Transformation	7
6.2	Observer Reference Elevation	7
6.3	Azimuth Binning and Horizon Angle	8
6.4	Curvature Correction (Lunar)	8
7	Data Acquisition	8
7.1	Lunar DEM — USGS LRO WAC GLD100	8
7.2	Earth DEM — ALOS World 3D 30 m (AW3D30)	9
8	Dependencies	10
9	Known Limitations and Future Work	10
A	Earth vs. Moon: Computational Differences	10
A.1	Body Parameters	10
A.2	Degree-to-Meter Conversion	11
A.3	ENU Frame and Ellipsoid	11
A.4	Observer Reference Elevation	11

A.5	Curvature Correction	12
A.6	Minimum Radius Guard	12
A.7	Return Signature of <code>max_elev_enu</code>	12

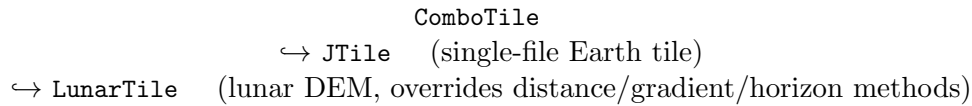
1 Overview

This module provides three classes for working with raster Digital Surface Models:

- **ComboTile** — base class that assembles a mosaic of ALOS PALSAR-2 30 m DSM tiles, exposes coordinate transforms, computes terrain gradients, and implements ENU-frame horizon-angle analysis.
- **JTile** — lightweight subclass for loading a single DSM file without mosaicking.
- **LunarTile** — subclass that re-implements distance and gradient computations for the Moon (using the IAU lunar ellipsoid via `pymap3d`) and applies a curvature correction to elevation angles.

The central scientific product is a *horizon profile*: for each azimuth bin around an observer, the maximum elevation angle to the terrain.

2 Class Hierarchy



JTile inherits all analysis methods from **ComboTile** unchanged. **LunarTile** overrides `get_lon_lat`, `compute_gradient`, `plot_map`, and `max_elev_enu`.

3 ComboTile — Core Terrain Tile

3.1 Constructor

```
1 ComboTile(line, root_dir='', center=[])
```

Listing 1: ComboTile initialisation signature

Parameters

- `line` — coordinate string with format "LAT_DEG DIR / LON_DEG DIR", e.g. "037 N/137 E".
- `root_dir` — path to the directory containing the ALOS DSM `.tif` files.

Behaviour

1. Calls `surrounding_tiles` to enumerate a 5×5 grid of tile coordinates centred on the parsed location.
2. Opens each matching `ALPSMLC30*_DSM.tif` file; files that are missing are skipped with a warning.
3. Merges all successfully opened sources into a single mosaic using `rasterio.merge.merge`.
4. Stores `self.elevations` (2-D numpy array), `self.transform` (affine), `self.shape`, and `self.center` (lon, lat of mosaic geometric midpoint).

3.2 Coordinate Utilities

3.2.1 `surrounding_tiles`

```

1 surrounding_tiles(lat_center, lon_center, tile_size=1.0, grid_size=5)
2 # Returns: list of (lat, lon) integer pairs

```

Listing 2: Tile grid generation

Produces a `grid_size`×`grid_size` grid of integer (lat, lon) pairs by offsetting from the centre in steps of `tile_size` degrees.

3.2.2 get_lon_lat

Returns two 2-D arrays of the same shape as `self.elevations` containing the longitude and latitude (in degrees) of every pixel centre. Uses `rasterio.transform.xy` applied to a meshgrid of row/column indices.

3.2.3 xy_to_lonlat / lonlat_to_xy

```

1 xy_to_lonlat(x, y)      # pixel (col, row) -> (lon, lat)
2 lonlat_to_xy(lon, lat) # (lon, lat)      -> (col, row)

```

Thin wrappers around `rasterio.transform.xy` and `rasterio.transform.rowcol` respectively.

3.2.4 get_elevation

Returns a 3-element array [lat, lon, elev] for a single pixel at (row, col).

3.3 Gradient and Slope

3.3.1 compute_gradient

```

1 grad_x, grad_y = tile.compute_gradient()

```

Listing 3: Gradient computation

Converts degree-per-pixel transform coefficients to metres using the spherical-Earth approximation at the mean tile latitude:

$$\Delta x_m = 111,320 \cos(\bar{\phi}) \Delta \lambda_{\text{deg}}, \quad (1)$$

$$\Delta y_m = 111,320 \Delta \phi_{\text{deg}}, \quad (2)$$

then applies `numpy.gradient` to obtain $\partial z / \partial x$ and $\partial z / \partial y$ in m m^{-1} .

3.3.2 plot_slope

```

1 tile.plot_slope(want_log=False, ax=None)

```

Derives slope magnitude $\|\nabla z\|$ and slope angle $\alpha = \arctan(\|\nabla z\|)$, then renders with `rasterio.plot.show`. Accepts an optional `matplotlib` axes object for subplot embedding.

3.4 Horizon / Sky-View Analysis

3.4.1 max_elev_enu

```

1 az_centers, max_elev, max_r, max_z, max_coords = tile.max_elev_enu(
2     bin_width=10.0,
3     radius_max_meters=20000,
4     radius0_meters=200,
5     lon0=lon, lat0=lat)

```

Listing 4: ENU-frame horizon profile with explicit observer

Requires explicit `lon0/lat0` (the observer location). The algorithm:

1. Calls `pymap3d.geodetic2enu` to convert all pixels to the local East-North-Up frame centred on the observer.
2. Estimates the observer’s reference elevation `elref` by cubic (falling back to linear/median) interpolation of pixels within `radius0_meters`.
3. Computes azimuth $\theta = \text{atan2}(E, N)$ (North-zero, East-positive convention) and horizontal distance $r = \sqrt{E^2 + N^2}$.
4. Extends the $[-180^\circ, 180^\circ]$ azimuth range to $[0^\circ, 720^\circ]$ by appending a $+360^\circ$ copy, ensuring seamless wraparound at $0^\circ/360^\circ$.
5. Bins by azimuth (bin centres from 0° to 360° in steps of `bin_width`) and selects the maximum elevation angle per bin.

Note: `self.center` is the geometric midpoint of the mosaic bounding box, *not* a physically meaningful observation site. Always pass `lon0` and `lat0` explicitly.

3.5 Visualisation

3.5.1 `plot_map`

Renders the elevation raster using `rasterio.plot.show` with the `plasma` colormap, attaches a labelled colorbar, and accepts an optional `ax` argument for multi-panel figures.

3.6 Data Export

3.6.1 `map_table`

```

1 table = tile.map_table(want_grad=True)
2 # Returns: (N, 7) array: [lat, lon, elev, grad_x, grad_y, slope,
   slope_deg]

```

Vectorised construction of a flat table of per-pixel terrain attributes.

3.6.2 `save_data`

```

1 tile.save_data("output.h5", compute_gradients=True)

```

Constructs a `TerrainData` object and serialises it to HDF5 via `TerrainData.save_to_h5`.

4 JTile — Single-File Earth Tile

```
1 tile = JTile("037 N/137 E", root_dir="ALOS_Data")
```

Listing 5: JTile initialisation

JTile is a minimal subclass of ComboTile that loads a *single* DSM file rather than assembling a mosaic. The file path is constructed as:

$$\text{root_dir}/\text{ALPSMLC30}_{\{\text{DIR}\}}\{\text{LAT}\}\{\text{DIR}\}\{\text{LON}\}_{\text{DSM.tif}}$$

All analysis methods (`compute_gradient`, `max_elev_enu`, etc.) are inherited from ComboTile without modification.

5 LunarTile — Lunar DEM Tile

5.1 Constructor

```
1 tile = LunarTile("lunar_-20.5_-43.5.tif", root_dir="Lunar_Test_Data")
```

Listing 6: LunarTile initialisation

Filename format: `lunar_{LAT}_{LON}.tif`. The constructor:

- Opens the file and stores the geographic bounds directly in degrees (no mosaic assembly).
- Pre-computes `_lon_grid` and `_lat_grid` arrays using the raster’s affine transform.
- Stores the lunar mean radius $R_{\text{moon}} = 1,737,400$ m for curvature corrections.
- Computes `geo_resolution` (degrees per pixel in each axis).

5.2 Overridden Methods

5.2.1 `get_lon_lat`

Returns the pre-computed `_lon_grid` and `_lat_grid` arrays directly, avoiding redundant calls to `rasterio.transform.xy`.

5.2.2 `compute_gradient`

Uses the lunar radius to convert degrees to metres at the tile’s mean latitude:

$$\Delta x_m = R_{\text{moon}} \cdot \cos(\bar{\phi}) \cdot |\text{transform.a}| \cdot \frac{\pi}{180}, \quad (3)$$

$$\Delta y_m = R_{\text{moon}} \cdot |\text{transform.e}| \cdot \frac{\pi}{180}, \quad (4)$$

where $\bar{\phi}$ is the centre latitude of the tile and $R_{\text{moon}} = 1,737,400$ m. Gradient components $\partial z/\partial x$ and $\partial z/\partial y$ are then computed with `numpy.gradient`.

5.2.3 `plot_map`

Uses `matplotlib.imshow` instead of `rasterio.plot.show`, passing the geographic extent in degrees. Masks `nodata` values before plotting.

5.2.4 max_elev_enu

```

1 el_deg, az_deg, az_centers, max_elev, max_r, max_z, max_coords = \
2     tile.max_elev_enu(bin_width=10.0, radius_max_meters=100000,
3                       radius0_meters=1000, lon0=lon, lat0=lat)

```

Listing 7: Lunar ENU horizon with curvature correction

Key differences from the Earth version:

1. **Lunar ellipsoid:** passes `pm.Ellipsoid.from_name('moon')` to `pymap3d.geodetic2enu`, using the IAU reference shape instead of GRS-80.
2. **Observer elevation:** determined by bilinear interpolation on the four surrounding pixels (seed), followed by linear `scipy.interpolate.griddata` refinement within `radius0_meters`.
3. **Curvature correction:** the elevation angle uses the corrected vertical component

$$dz_{\text{corr}} = u + \frac{r^2}{2R_{\text{moon}}}, \quad (5)$$

where u is the ENU “up” component and r is horizontal distance, to account for lunar surface curvature over long baselines.

4. A minimum radius guard ($r > 500$ m) excludes immediate neighbours that may produce artefactual elevation angles.

Note that this override also returns two additional leading arrays (`el_deg`, `az_deg`) compared to the `ComboTile` version, giving the per-pixel elevation and azimuth arrays before binning.

6 Key Algorithms

6.1 ENU Frame Transformation

The function `pymap3d.geodetic2enu` converts geodetic coordinates (ϕ, λ, h) to the local East-North-Up frame centred at (ϕ_0, λ_0, h_0) :

$$\begin{pmatrix} E \\ N \\ U \end{pmatrix} = \mathbf{R}_{\text{ENU}}(\phi_0, \lambda_0) \left[\mathbf{r}_{\text{ECEF}}(\phi, \lambda, h) - \mathbf{r}_{\text{ECEF}}(\phi_0, \lambda_0, h_0) \right], \quad (6)$$

where $\mathbf{R}_{\text{ENU}}$ is the standard rotation matrix from ECEF to ENU coordinates and the ECEF positions are computed using the selected ellipsoid (Earth GRS-80 or Moon IAU 2015).

6.2 Observer Reference Elevation

To avoid biasing the horizon by placing the observer above or below the surface, `elref` is estimated in two stages:

Stage 1 (bilinear seed):

$$h_{\text{seed}} = \sum_{(r,c) \in \{0,1\}^2} e_{r,c} (1 - dc)^{1-c} dc^c (1 - dr)^{1-r} dr^r, \quad (7)$$

where dc , dr are the fractional column/row offsets of the observer within its pixel.

Stage 2 (linear griddata): Using pixels within `radius0_meters` (in the seed ENU frame), `scipy.interpolate.griddata` with `method='linear'` interpolates to $(E, N) = (0, 0)$. If fewer than 3 pixels are available, the seed is used directly.

6.3 Azimuth Binning and Horizon Angle

Given the ENU-projected pixel array, binning proceeds as:

1. Compute azimuth $\theta_i = \text{atan2}(E_i, N_i)$ and horizontal distance $r_i = \sqrt{E_i^2 + N_i^2}$.
2. Extend the $[-180^\circ, 180^\circ]$ azimuth range to $[0^\circ, 720^\circ]$ by appending a $+360^\circ$ copy to handle the $0^\circ/360^\circ$ wraparound.
3. Compute elevation angle $\epsilon_i = \arctan(dz_i/r_i)$.
4. Iterate over bin centres from 0° to 360° in steps of `bin_width` and select the pixel with the maximum ϵ in each bin.

6.4 Curvature Correction (Lunar)

Over long baselines the curvature of the lunar surface raises the terrain relative to a flat-tangent-plane reference. The correction to the “up” component is:

$$dz_{\text{corr}} = u + \frac{r^2}{2R_{\text{moon}}}, \quad (8)$$

which is the standard flat-to-spherical horizon correction (first-order in r/R). For $r = 100$ km and $R_{\text{moon}} = 1737.4$ km this amounts to approximately $+2.9$ km, a non-negligible effect for large-radius analyses.

7 Data Acquisition

This section describes how to obtain the elevation data required by each tile class.

7.1 Lunar DEM — USGS LRO WAC GLD100

The `LunarTile` class reads GeoTIFF files clipped from the USGS Lunar Reconnaissance Orbiter (LRO) WAC Global Lunar DTM (GLD100) at 100 m/pixel. The full global mosaic is publicly accessible via USGS Astrogeology’s HTTP server and can be clipped to any geographic area in a single `gdalwarp` command without downloading the full dataset.

Dataset reference: https://astrogeology.usgs.gov/search/map/moon_lroc_wac_dtm_gld100_118m

Command:

```
1 gdalwarp \
2   -te -2.5 -2.5 2.5 2.5 \
3   -te_srs "+proj=longlat +a=1737400 +b=1737400 +no_defs" \
4   -t_srs "+proj=longlat +a=1737400 +b=1737400 +no_defs" \
5   /vsicurl/https://planetarymaps.usgs.gov/mosaic/
6   Lunar_LRO_WAC_GLD100_DTM_79S79N_100m_v1.1.tif \
   Lunar_Test_Data/lunar_0.0_0.0.tif
```

Listing 8: Clip a region from the USGS lunar mosaic

Argument breakdown:

- **Line 1** (`gdalwarp`) — invokes the GDAL warping utility. No resampling or reprojection is needed here; the command is used purely to clip the remote mosaic to a local file.

- **Line 2** (`-te`) — sets the target extent as `xmin ymin xmax ymax` in the coordinate system given by `-te.srs`. The example clips a $5^\circ \times 5^\circ$ box centred on 0°N , 0°E . Adjust these four values to your area of interest.
- **Line 3** (`-te.srs`) — defines the CRS of the bounding-box coordinates supplied to `-te`. A custom geographic CRS is specified inline using the lunar sphere radius ($a = b = 1,737,400\text{ m}$); this is necessary because the USGS mosaic uses a lunar body-fixed frame rather than a terrestrial datum.
- **Line 4** (`-t.srs`) — sets the output CRS. Using the same lunar geographic CRS as `-te.srs` ensures the saved file retains geographic (degree) coordinates, which is what `LunarTile` expects.
- **Line 5** (`/vsicurl/...`) — the remote data source. The `/vsicurl/` prefix instructs GDAL to stream the file over HTTP without downloading it in full; only the tiles intersecting the requested extent are transferred.
- **Line 6** — local output path. Follow the naming convention `lunar_{LAT}_{LON}.tif` (e.g. `lunar_-20.5_-43.5.tif`) so that `LunarTile` can parse the centre coordinates from the filename.

Tip: `gdalwarp` with `/vsicurl/` streams only the requested spatial subset. For small areas ($\lesssim 20^\circ$ across) this is fast; for very large clipping extents consider downloading the full mosaic first.

7.2 Earth DEM — ALOS World 3D 30 m (AW3D30)

The `ComboTile` and `JTile` classes expect individual 1-degree tiles in JAXA's `ALPSMLC30_*.DSM.tif` format. Unlike the lunar dataset these tiles are *not* available via a single public streaming URL; they must be downloaded individually from JAXA after free registration.

Steps:

1. Register for a free account at https://www.eorc.jaxa.jp/ALOS/en/dataset/aw3d30/aw3d30_e.htm.
2. After activation, use the JAXA tile browser or bulk FTP path to download the 1-degree tile zip archives for your area. Each zip contains a file named `ALPSMLC30_{DIR}_{LAT}_{DIR}_{LON}.DSM.tif`, e.g. `ALPSMLC30_N037E137_DSM.tif`.
3. Extract the `.DSM.tif` files into a common directory and point `root_dir` at that directory when constructing a `ComboTile` or `JTile`.

Alternatively, the AW3D30 dataset is mirrored by OpenTopography and can be fetched programmatically via their S3 endpoint (no authentication required, but the bucket is requester-pays):

```

1 # List available tiles
2 aws s3 ls s3://raster/AW3D30/ \
3   --endpoint-url https://opentopography.s3.sdsc.edu \
4   --no-sign-request
5
6 # Download a single tile
7 aws s3 cp s3://raster/AW3D30/N037/ALPSMLC30_N037E137_DSM.tif ./
8   ALOS_Data/ \
9   --endpoint-url https://opentopography.s3.sdsc.edu \
   --no-sign-request

```

Listing 9: Download individual tiles via OpenTopography S3

Note: ComboTile constructs a 5×5 mosaic of surrounding tiles automatically. For an observer at 37°N 137°E you will need the tiles from 34°N – 41°N and 134°E – 141°E (25 tiles total). Download all tiles you expect to be needed before constructing the mosaic; missing tiles are skipped with a warning and the mosaic will cover only the available subset.

8 Dependencies

Package	Import alias	Purpose
rasterio	—	DSM file I/O, mosaic merging, affine transforms
numpy	np	Array operations, gradient, meshgrid
matplotlib	plt	Raster and profile visualisation
pymap3d	pm	Geodetic-to-ENU conversion; lunar ellipsoid
scipy.interpolate	spi	griddata for observer elevation estimation
h5py	—	HDF5 serialisation of TerrainData
pandas	—	(Imported; not directly used in current methods)

pymap3d version note: `pm.Ellipsoid.from_name('moon')` requires `pymap3d ≥ 3.0`. On earlier versions, define the ellipsoid manually:

```
1 lunar_ell = pm.Ellipsoid(semimajor_axis=1738100, semiminor_axis
    =1736000)
```

9 Known Limitations and Future Work

- **Nodata handling:** ComboTile does not mask `nodata` pixels before computing gradients or elevation angles. Nodata values (often -9999) can produce large spurious angles in the horizon profile.
- **save_data gradient signature:** `compute_gradient` currently returns two values (`grad_x`, `grad_y`), but `save_data` unpacks four values. This will raise a `ValueError` at runtime; the `slope` and `slope_deg` computations need to be moved inside `save_data` or `compute_gradient` must be updated to return all four quantities.
- **Lunar curvature correction:** The $r^2/(2R)$ correction is first-order in r/R . For $r > 200$ km the second-order term becomes non-negligible and a full spherical treatment is recommended.

A Earth vs. Moon: Computational Differences

Although ComboTile and LunarTile share the same public API, several underlying computations differ substantially between the two bodies. This appendix collects those differences in one place for reference.

A.1 Body Parameters

Parameter	Earth (GRS-80)	Moon (IAU 2015)
Equatorial radius a	6,378,137 m	1,738,100 m
Polar radius b	6,356,752 m	1,736,000 m
Flattening f	$1/298.257$	$\approx 1/825$
Ellipsoid used by pymap3d	wgs84 (default)	<code>pm.Ellipsoid.from_name('moon')</code>

A.2 Degree-to-Meter Conversion

Both classes compute terrain gradients by converting degree-per-pixel transform coefficients to physical meters. The conversion constants differ because of the different body radii.

Earth (`ComboTile.compute_gradient`):

$$\Delta x_m = 111,320 \cos(\bar{\phi}) |\Delta \lambda_{\text{deg}}|, \quad (9)$$

$$\Delta y_m = 111,320 |\Delta \phi_{\text{deg}}|, \quad (10)$$

where the factor $111,320 \text{ m deg}^{-1}$ is the standard spherical-Earth approximation ($\pi R_{\oplus}/180$) evaluated at the mean Earth radius.

Moon (`LunarTile.compute_gradient`):

$$\Delta x_m = \frac{\pi R_{\text{moon}}}{180} \cos(\bar{\phi}) |\Delta \lambda_{\text{deg}}|, \quad (11)$$

$$\Delta y_m = \frac{\pi R_{\text{moon}}}{180} |\Delta \phi_{\text{deg}}|, \quad (12)$$

with $R_{\text{moon}} = 1,737,400 \text{ m}$, giving a conversion factor of approximately $30,341 \text{ m deg}^{-1}$ — roughly 27% of the Earth value.

A.3 ENU Frame and Ellipsoid

The function `pymap3d.geodetic2enu` is used by both classes to project pixels into a local East-North-Up coordinate frame centred on the observer. The only difference is the ellipsoid passed:

Class	Ellipsoid argument
<code>ComboTile</code>	(none — defaults to WGS-84 / GRS-80)
<code>LunarTile</code>	<code>ell=pm.Ellipsoid.from_name('moon')</code>

Using the correct ellipsoid matters for the accuracy of the ENU projection: a point at 100 km horizontal distance would be misplaced by several kilometres if the Earth ellipsoid were used for the Moon.

A.4 Observer Reference Elevation

Both classes estimate the observer’s ground elevation (`elref`) by interpolating the DEM near the observer location. The method differs slightly:

	Earth (<code>ComboTile</code>)	Moon (<code>LunarTile</code>)
Seed interpolation	Direct ENU at altitude 0 m; cubic <code>griddata</code> within <code>radius0_meters</code> , falling back to linear then median.	Bilinear interpolation on the four surrounding pixels first; then linear <code>griddata</code> within <code>radius0_meters</code> , falling back to the bilinear seed.
Minimum sample count	≥ 4 for cubic, ≥ 1 for median fallback.	≥ 3 for linear, seed used otherwise.

The lunar approach uses a bilinear seed first because the larger pixel footprint (100 m vs. 30 m) means the observer is more likely to lie well within a single pixel, making the four-corner weighted average a reliable and cheap starting point before the `griddata` refinement step.

A.5 Curvature Correction

Over long baselines the surface of a spherical body curves away from the observer’s local tangent plane. `ComboTile` ignores this effect; `LunarTile` applies a first-order correction:

Class	Vertical component used for elevation angle
<code>ComboTile</code>	$dz = u$ (ENU “up” component directly)
<code>LunarTile</code>	$dz = u + \frac{r^2}{2R_{\text{moon}}}$

The correction term $r^2/(2R)$ represents the height by which the spherical surface rises above the tangent plane at horizontal distance r . Its magnitude for both bodies is:

r	Earth ($R = 6371$ km)	Moon ($R = 1737$ km)
10 km	≈ 7.8 m	≈ 29 m
50 km	≈ 196 m	≈ 720 m
100 km	≈ 785 m	$\approx 2,880$ m

The correction is omitted for Earth on the grounds that the default analysis radius (`radius_max_meters=20000`) keeps $r \leq 20$ km, where the effect (< 32 m) is small relative to the 30 m pixel size. For the Moon the default radius is 100 km, where ignoring curvature would introduce a ~ 2.9 km error in the apparent terrain height — a horizon-angle error of order $\arctan(2900/100,000) \approx 1.7^\circ$, which is non-negligible for illumination or line-of-sight analyses.

A.6 Minimum Radius Guard

`LunarTile.max_elev_enu` excludes pixels within 500 m of the observer (`r > 500`), whereas `ComboTile.max_elev_enu` only excludes the observer pixel itself (`r > 0`). The larger guard radius on the Moon reflects the coarser pixel footprint (100 m) and prevents the steeply-angled immediate neighbours from dominating the near-field bins of the horizon profile.

A.7 Return Signature of `max_elev_enu`

The two implementations return different numbers of arrays:

Class	Return values
<code>ComboTile</code>	<code>az_centers</code> , <code>max_elev</code> , <code>max_r</code> , <code>max_z</code> , <code>max_coords</code>
<code>LunarTile</code>	<code>el_deg</code> , <code>az_deg</code> , <code>az_centers</code> , <code>max_elev</code> , <code>max_r</code> , <code>max_z</code> , <code>max_coords</code>

`LunarTile` additionally returns the per-pixel elevation angle array (`el_deg`) and azimuth array (`az_deg`) before binning. These are useful for diagnostic plots and for verifying the curvature correction.